

VyaparFlow

Agentic AI Design, RAG and Voice Pipeline

Transcript

LangGraph Supervisor, Agents, Tools, Memory, Prompts, Document Retrieval, Whisper, IndicTrans2, and TTS

This transcript explains the intelligence layer of VyaparFlow. It covers the LangGraph-based agentic design, supervisor orchestration, specialized business agents, tool calling, memory, prompt design, document ingestion, embeddings, retrieval, multilingual voice processing, translation, and text-to-speech response generation.

Agent Framework	LangGraph supervisor graph with specialized domain agents
Core Agents	Supervisor, Inventory, Finance, Business Advisor, Customer/Reminder, RAG Knowledge Agent
Tools	Safe backend tools for product lookup, stock updates, sale recording, analytics, reminders, audit logging
Memory	Short-term command state, long-term business memory, vector memory, audit trail
RAG Pipeline	Document ingestion, chunking, embeddings, vector search, reranking, grounded answer generation
Voice Pipeline	Audio capture, Whisper STT, language detection, IndicTrans2 translation, LLM response, TTS

1. Opening Context

Presenter:

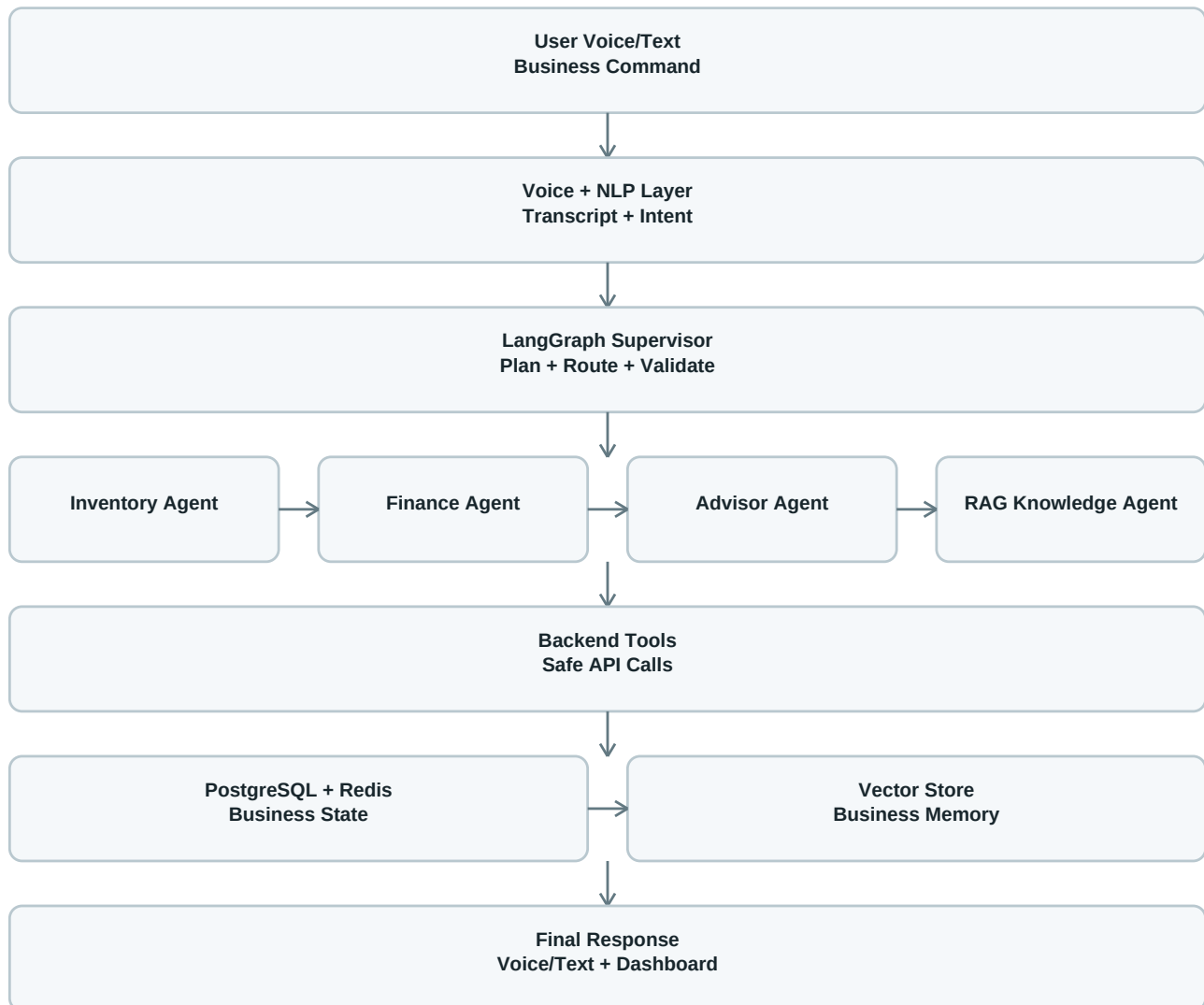
In this transcript, I will explain the AI intelligence layer of VyaparFlow. The project is not only a voice interface over a database. Its real value comes from agentic workflow automation, where a user gives one natural command and multiple AI agents coordinate to complete inventory, finance, analytics, reminder, and advisory tasks.

Presenter:

For this design, LangGraph is a strong choice because it lets us model the AI workflow as a graph. Each node can represent a supervisor decision, an agent action, a tool call, a validation step, or a clarification step. This is better than a simple one-shot LLM call because business workflows need state, branching, retries, and deterministic tool execution.

2. Agentic AI Architecture

LangGraph Agentic Architecture



Presenter:

The architecture starts with the user command. The voice and NLP layer converts it into structured input. The LangGraph Supervisor receives this input and decides the next node. It may call the Inventory Agent, Finance Agent, Advisor Agent, or RAG Knowledge Agent. These agents do not write directly to the database. They call safe backend tools, and those tools use domain services to validate and persist changes.

3. LangGraph Supervisor Design

- Receives normalized command state from the voice/NLP pipeline.
- Classifies the workflow type: sale, expense, stock update, report, reminder, advisory question, or document question.
- Chooses which agent should act next.
- Maintains graph state, including transcript, language, intent, entities, confidence, tool results, errors, and final response.
- Routes to clarification when product, quantity, price, or intent is uncertain.
- Routes to validation before any business-critical write.
- Routes to final response generation after tools complete successfully.

Graph State:

```
command_id: UUID
business_id: UUID
user_id: UUID
transcript: str
source_language: str
normalized_text: str
intent: str
entities: dict
confidence: float
plan: list[WorkflowStep]
tool_results: list[ToolResult]
memory_context: list[RetrievedContext]
requires_confirmation: bool
final_response: str
```

4. Agent Roles

Agent	Responsibility	Typical Tools
Supervisor Agent	Plans workflow, routes agents, handles branching, errors, and confirmation.	route_intent, validate_plan, finalize_response
Inventory Agent	Handles product lookup, stock deduction/addition, low-stock detection, restock suggestions.	find_product, deduct_stock, add_stock, check_low_stock
Finance Agent	Records sales, expenses, payments, dues, profit and cash-flow updates.	record_sale, record_expense, update_payment_status
Business Advisor Agent	Generates insights, trends, pricing suggestions, weekly summaries, demand forecasts.	get_analytics, generate_insight, forecast_demand
Customer/Reminder Agent	Handles customer dues, follow-up reminders, supplier reminders, low-stock alerts.	find_customer, create_reminder, list_due_customers
RAG Knowledge Agent	Answers questions from business documents, invoices, policy docs, schemes, and help content.	retrieve_documents, rerank_context, grounded_answer
Audit/Safety Agent	Checks risky actions, records reasoning trail, enforces confirmation rules.	audit_step, check_policy, require_confirmation

5. Tool Calling Design

Agents should call tools through a controlled registry. A tool is a safe wrapper over backend service methods, not direct database access.

Tool	Input	Output	Safety Rule
inventory.find_product	business_id, product_name	matched products	Tenant check and confidence threshold.
inventory.deduct_stock	product_id, quantity, command_id	new stock level	Idempotent, transactional, stock validation.
finance.record_sale	items, amount, payment_status	transaction id	Validated price, quantity, business ownership.

Tool	Input	Output	Safety Rule
finance.record_expense	category, amount, note	expense id	Positive amount and allowed category.
analytics.get_summary	business_id, date_range	dashboard metrics	Read-only.
reminder.create	message, due_at, type	reminder id	Deduplicate similar reminders.
rag.retrieve	query, filters	ranked chunks	Business-specific access control.
audit.log_agent_step	command_id, agent, action, result	audit id	Append-only.

6. Memory Design

Presenter:

VyaparFlow needs memory at multiple levels. Short-term memory is the current command state inside LangGraph. Conversation memory stores the current session. Long-term business memory stores stable facts such as common products, preferred language, suppliers, customer dues, and recurring business patterns. Vector memory stores embedded documents and past insights for retrieval.

Memory Type	Stored In	Purpose
Graph State Memory	LangGraph state object	Keeps current workflow state, steps, tool results, and errors.
Session Memory	Redis	Tracks active user session, current command, pending confirmation.
Business Memory	PostgreSQL	Products, customers, preferences, recurring actions, historical transactions.
Vector Memory	pgvector or vector DB	Semantic retrieval from invoices, documents, notes, insights, prior reports.
Audit Memory	PostgreSQL audit_logs	Trace decisions, tool calls, risky actions, confirmations.
Prompt Memory	Prompt templates/version store	Keeps stable system prompts and versioned agent instructions.

7. Prompt Design

- Use separate prompts for supervisor routing, each agent, structured extraction, grounded RAG answering, and final user response.
- Prompts must include business safety rules: do not invent transactions, do not skip validation, ask clarification when confidence is low.
- Tool prompts should force structured JSON outputs so Pydantic can validate the result.
- Final response prompts should be short, conversational, and in the user's preferred language.
- RAG prompts should cite retrieved chunks internally and answer only from retrieved context when the question is document-specific.

Supervisor Prompt Skeleton:

You are VyaparFlow Supervisor Agent.

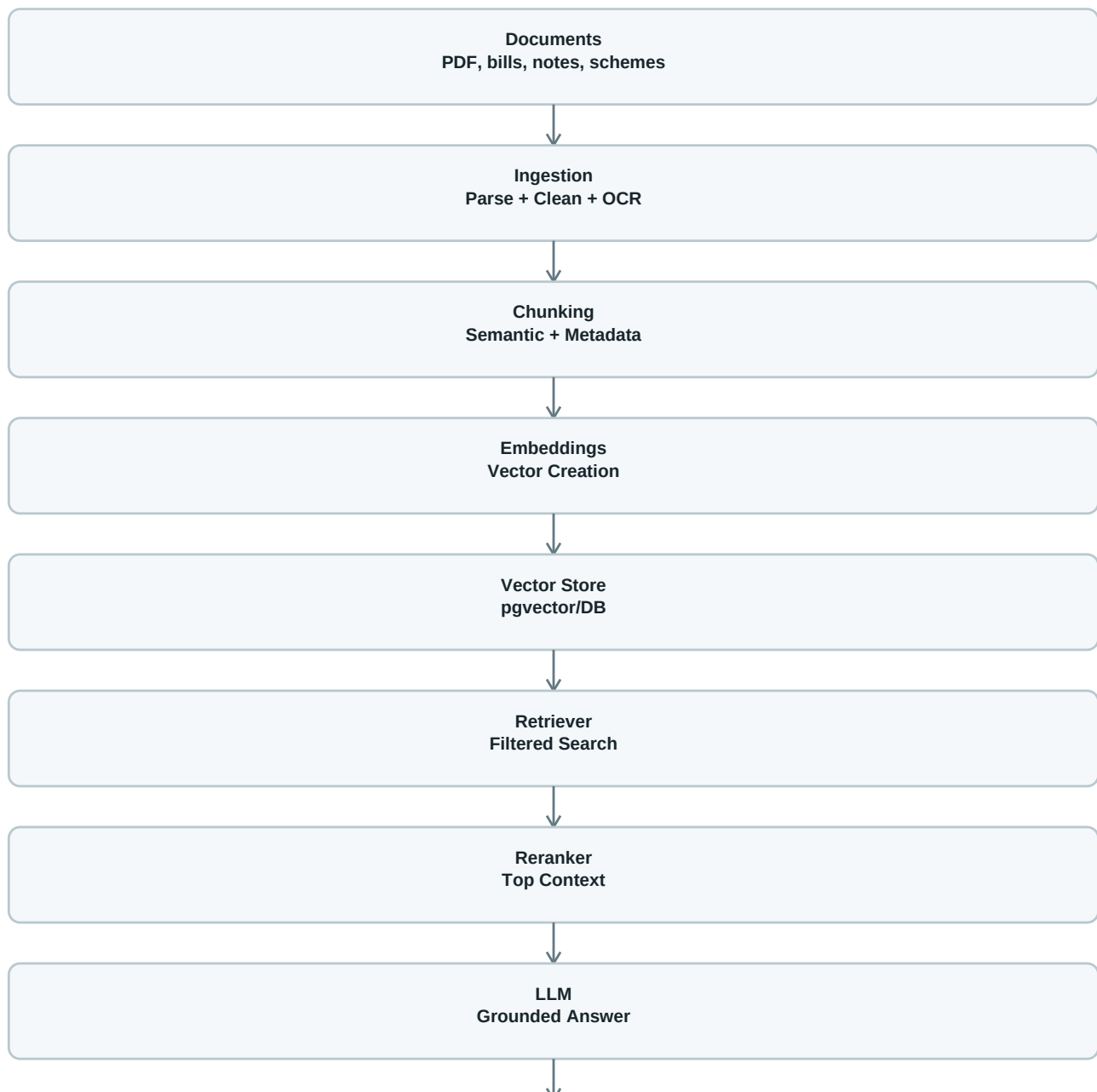
Goal: convert a user business command into a safe workflow plan.

Rules:

1. Identify intent and required agents.
2. Never write data directly; call only registered tools.
3. Ask clarification if product, quantity, price, customer, or date is uncertain.
4. Require confirmation for risky actions.
5. Return structured JSON with next_agent, required_tools, and reason.

8. RAG Pipeline Overview

RAG Pipeline



9. Document Ingestion

- Accept PDFs, images, invoices, OCR bills, business notes, help documents, and future government scheme documents.
- Extract text using PDF parsers for digital PDFs and OCR for scanned bills/images.
- Clean extracted text by removing repeated headers, footers, page numbers, noise, and broken spacing.
- Attach metadata such as business_id, document_type, source_file, upload_date, language, page_number, and access level.
- Store original files in object storage and extracted text/chunks in PostgreSQL.

10. Chunking and Embeddings

- Use semantic chunking for documents with headings and sections.
- Use smaller chunks for invoices and bills because line-item precision matters.
- Store chunk text, metadata, embedding vector, source document id, and page number.
- Use multilingual embeddings if documents may contain Hindi, Hinglish, or regional language content.
- For MVP, pgvector inside PostgreSQL is enough; a managed vector database can be used later if retrieval scale grows.

Table	Important Fields
documents	id, business_id, file_url, document_type, language, status, uploaded_by, created_at
document_chunks	id, document_id, business_id, chunk_text, page_number, metadata_json, embedding_vector
retrieval_logs	id, business_id, query, retrieved_chunk_ids, response_id, created_at

11. Retrieval and Grounded Generation

- Convert user query into an embedding.
- Filter by business_id, document type, language, and access permissions.
- Retrieve top-k semantically similar chunks.
- Optionally rerank chunks using a cross-encoder or LLM-based reranker.
- Pass only relevant chunks to the LLM with a grounded answer prompt.
- If retrieved context is insufficient, say that the answer is not available in the uploaded documents.
- Log retrieval results for debugging and quality improvement.

Grounded RAG Prompt Skeleton:

Answer the user's question using only the provided context.

If the answer is not present, say you do not have enough information.

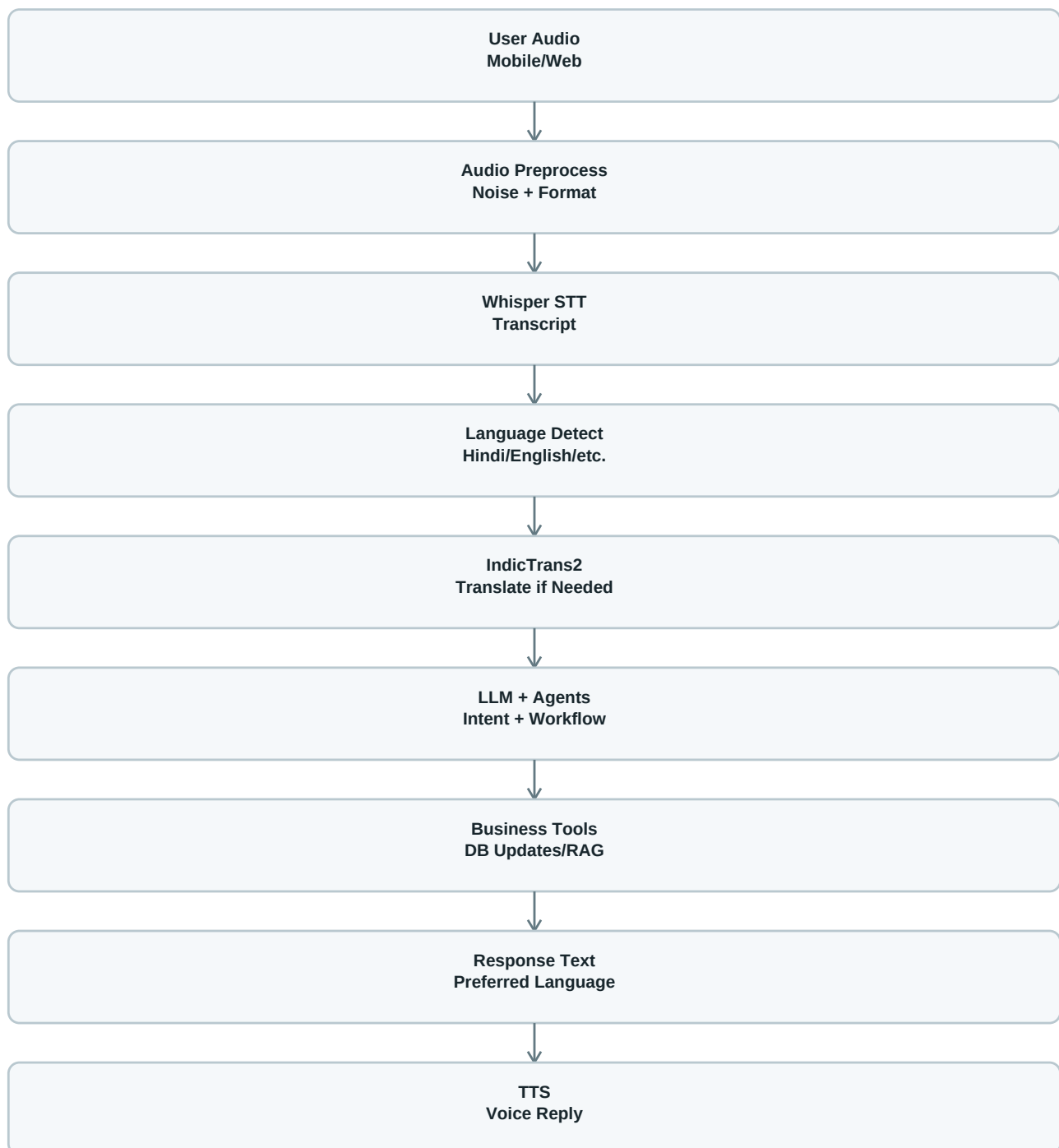
Do not invent policies, prices, dues, or financial facts.

Return:

- short_answer
- supporting_points
- source_chunk_ids
- confidence

12. Voice Pipeline Overview

Voice Pipeline



13. Whisper Speech-to-Text

Presenter:

For speech-to-text, Whisper can be used to transcribe user audio into text. Since the target users may speak naturally, sometimes in Hindi, English, Hinglish, or regional languages, the STT layer should return transcript, detected language, confidence, timestamps if available, and possible alternatives.

- Normalize audio format before sending it for transcription.
- Store raw audio only if the user permits audio retention.
- Use transcript confidence to decide whether to process automatically or ask the user to repeat.
- Keep the original transcript and normalized transcript separately.
- Log transcription latency and failure rate for monitoring.

14. IndicTrans2 Translation Layer

Presenter:

IndicTrans2 is useful when the user speaks in an Indian language but the LLM workflow is more reliable in English or a normalized internal language. The translation layer should preserve business entities carefully, such as product names, quantities, prices, dates, customer names, and units.

- Detect source language after STT.
- Translate transcript into the internal processing language when needed.
- Preserve named entities and numbers during translation.
- Store source transcript, translated transcript, and language metadata.
- Generate final response in the user's preferred language.

15. Text-to-Speech Response

- The final agent response should be short enough for voice playback.
- Use TTS for confirmations, summaries, low-stock alerts, and clarification questions.
- Prefer text plus voice response so users can read and listen.
- For sensitive financial details, let the user control whether responses are spoken aloud.
- Cache repeated static responses where useful, but generate dynamic transaction confirmations per command.

16. End-to-End Example

User says: 'Aaj maine 20 pickle bottles bechi, 100 rupaye each.'

- Audio is captured by the frontend and sent to FastAPI.
- Whisper converts speech to transcript.
- Language detection identifies Hindi/Hinglish.
- IndicTrans2 or normalization converts the command into internal English: 'Today I sold 20 pickle bottles for 100 rupees each.'
- Supervisor Agent identifies SALE intent.
- Inventory Agent finds pickle bottle product and checks stock.
- Finance Agent records sale amount of 2,000 rupees.
- Inventory tool deducts 20 units and creates stock movement.
- Analytics job updates dashboard metrics.
- Advisor checks low stock and may suggest restocking.
- TTS replies in the user's language: 'Sale recorded. 20 pickle bottles deducted. Revenue is 2,000 rupees.'

17. Safety, Validation, and Guardrails

- Never allow agents to directly write to database tables.
- Validate every AI-generated structured output using Pydantic schemas.
- Use tool allowlists for each agent.
- Require confirmation for low-confidence product matches, negative stock, unusually high amounts, or customer-facing messages.
- Use idempotency keys to avoid duplicate transactions from repeated audio submissions.
- Maintain audit logs for transcript, intent, tool calls, confirmations, and final result.
- Use business_id filters in every tool and retrieval query.
- Do not answer document questions without retrieved context.

18. Monitoring and Evaluation

Area	Metrics
Voice	STT latency, language detection accuracy, repeat-request rate, low-confidence rate.
Agent Workflow	Workflow success rate, clarification rate, tool failure rate, average steps per command.
RAG	Retrieval precision, no-answer rate, grounded answer quality, source chunk usage.
Business Tools	Transaction success, rollback count, duplicate command detection, low-stock alert accuracy.
Cost	STT minutes, LLM tokens, embedding count, TTS characters, vector storage.
User Experience	Command completion time, correction rate, dashboard update delay, user satisfaction.

19. Implementation Order

- Implement text-command agent workflow first before full voice automation.
- Create tool registry and connect Inventory, Finance, Analytics, Reminder, and Audit tools.
- Build LangGraph supervisor with routing, clarification, and final response nodes.
- Add Whisper transcription and language detection.
- Add IndicTrans2 translation or normalization layer for multilingual commands.
- Add TTS for final responses and clarification questions.
- Build document ingestion with parsing, cleaning, chunking, embeddings, and vector storage.
- Add RAG Knowledge Agent for document-grounded answers.
- Add monitoring, evaluation datasets, and audit dashboards.

20. Closing Summary

Presenter:

To summarize, the Agentic AI layer of VyaparFlow is designed around a LangGraph Supervisor that routes user commands to specialized agents. Each agent has a clear responsibility, and all business actions are performed through safe tools connected to backend services. Memory is divided into graph state, Redis session memory, PostgreSQL business memory, vector memory, and audit logs.

Presenter:

The RAG pipeline allows VyaparFlow to answer questions from uploaded business documents, invoices, reports, and future scheme documents. The voice pipeline allows users to speak naturally, using Whisper for transcription, language detection and IndicTrans2 for translation, LLMs and agents for workflow execution, and TTS for spoken responses.

Takeaway: VyaparFlow's intelligence layer combines LangGraph agents for action, RAG for grounded knowledge, and a multilingual voice pipeline for accessible business automation.

Recommended next step: create detailed sequence diagrams and prompt templates for each agent workflow.